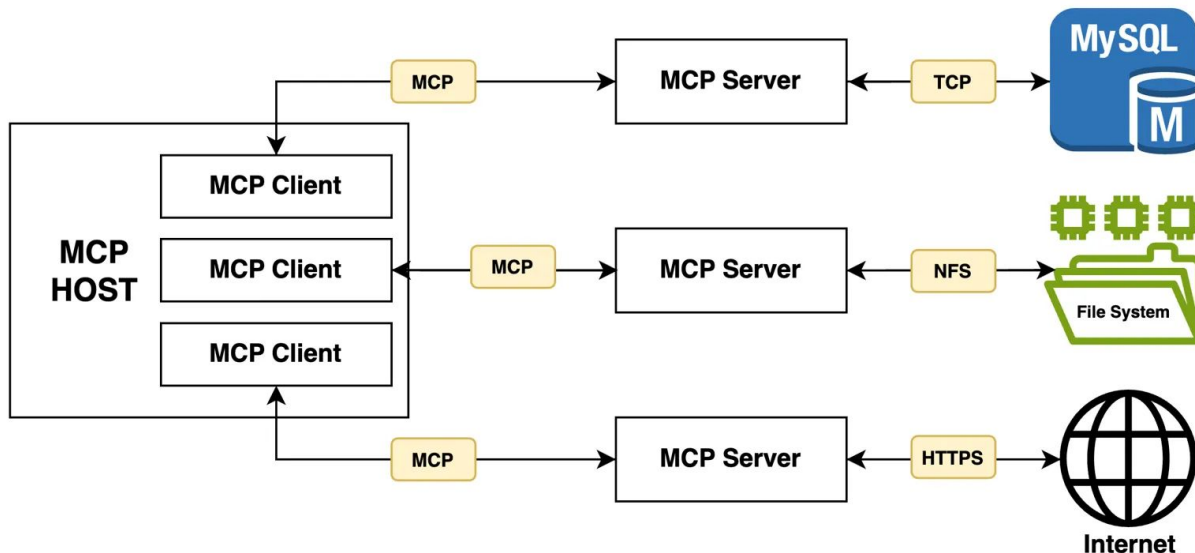


MCP Client

What is an MCP Client?

MCP (Model-Context Protocol) client are a component/interface that maintain 1:1 connections with servers and act as a software interface that connects to an MCP server to utilize AI tools and language models remotely. It allows you to **invoke multiple tools or models** in a single session, supporting **tool calling** and **workflow automation**.

It also provides a **standardized communication layer** between client applications and AI servers, enabling flexible integration.



Key Features:

- The MCP Client is a lightweight interface for connecting an LLM to an MCP-compatible tool server.
- It handles communication, context management, and tool execution requests/responses.
- Simplifies tool integration with LLMs.
- Enables structured multi-turn(conversation over multiple steps) interactions.
- Designed for efficiency, modularity, and extensibility.

MCP Hosts:

MCP Hosts are the AI applications like Claude Desktop or IDEs that manage the connection between the MCP servers and MCP clients. They ensure smooth communication and stable connections which provides AI models with right tools and context to perform their tasks effectively. This architecture helps AI models to access different data sources through a single unified protocol helps in eliminating need for custom connectors and helps in simplifying integration.

Key Components of MCP Client

- **Client Session:** Manages the session lifecycle between the client and the MCP server. Handles connection, tool invocation, and streaming responses.
- **Server Parameters:** Configuration settings required to connect to an MCP server (e.g., protocol type, address).
- **Tool Invocation Interface:** MCP Client sends structured requests to invoke tools (e.g., search, comparison, file access). Tools respond with structured outputs that the LLM can parse and act upon.
- **Context Management:** Maintains session state including tool outputs, errors, and execution .

Asyncio-await in MCP Clients

1. Tool Responses Can Take Time

- Your tools may take time to process (e.g., accessing an API, doing calculations).
- Instead of just *waiting* (which blocks everything), the MCP client uses `await` to pause and **do other work** while it waits for the tool's result.

2. Tools Use Streams (stdio)

- MCP tools are connected via **stdin/stdout**, like a chat with the tool.
- These are **I/O operations**, and in Python, I/O is best handled with `asyncio`.

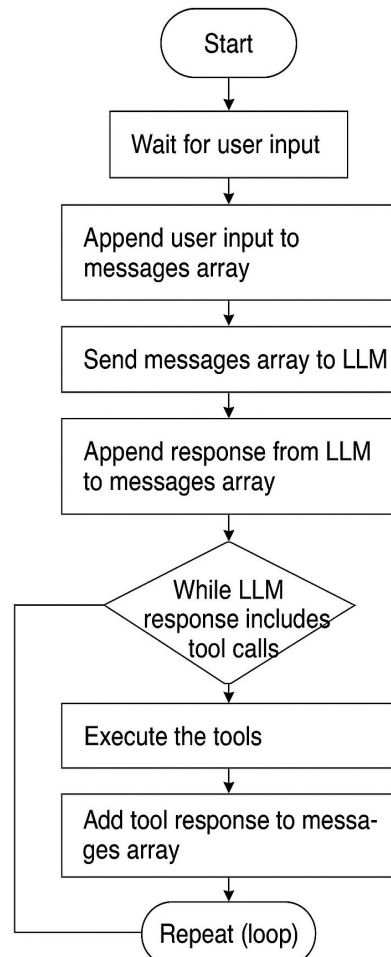
3. Multiple Users or Tasks at Once

- If more than one user interacts at the same time, `asyncio` lets the MCP client **handle many requests together**, without crashing or slowing down.

MCP Client Architecture Overview

Flow Explanation:

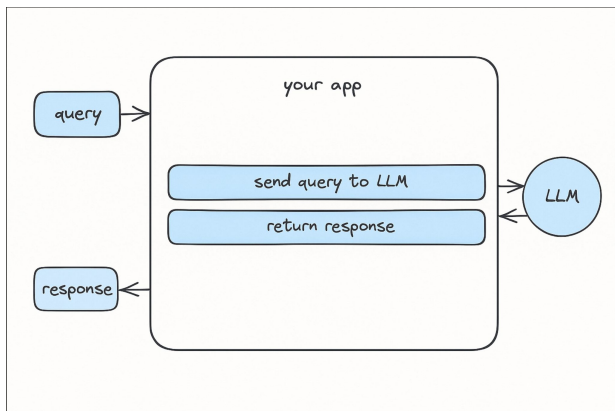
1. User provides input via a chat or interface.
2. LLM interprets the input and decides if a tool is needed.
3. MCP Client formats and sends the tool request.
4. MCP Tool Server executes the tool logic and returns results.
5. Results are passed back through the LLM for final response.



Difference between a Chatbot and MCP Client

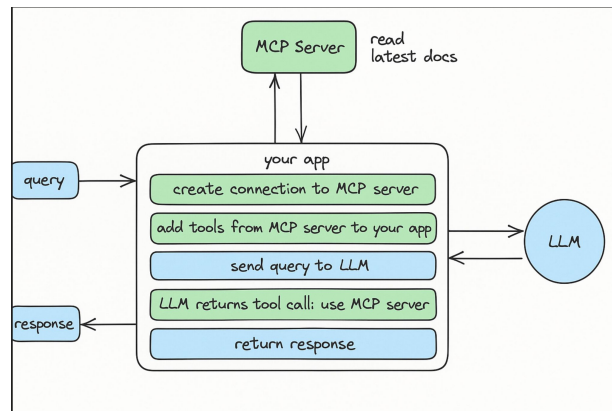
Chatbot

- A conversational AI system designed to interact with users, typically answering questions or performing tasks based on pre-trained data or internal logic.
- Primarily focused on user interaction through text or voice, providing responses based on trained data or simple API integrations.
- Uses HTTP/REST APIs or proprietary protocols, often requiring custom middleware for external communication.



MCP Client

- An application-side component that uses the Model Context Protocol to connect AI models with external tools, data sources, and services in a standardized way.
- Facilitates two-way, secure communication between AI models and external systems (e.g., APIs, databases, tools like Slack or GitHub)
- Uses standardized transport layers like STDIO (local) or HTTP+SSE (remote) for communication with MCP servers.



Agent Tool Calling VS MCP Tool Calling

Agent Tool Calling

- Tools are hardcoded inside the agent
- The agent directly manages and calls the tools
- Static – you give a fixed list of tools
- Needs manual filtering or logic

MCP Tool Calling

- Tools are registered and served by an MCP server
- MCP server manages tools; agent calls via MCP session
- Dynamic – agent loads tools from MCP at runtime
- Easily filter tools dynamically based on name, etc.

Workflow at code level

1. Tool Loading

- `load_mcp_tools()` from `langchain_mcp_adapters.tools` dynamically loads custom tools from `tools.py`.

2. Prompt Initialization

- System prompts are pulled from `prompts.py`, including role definitions and behavior instructions.

3. LLM Setup

- `ChatGroq` is used to initialize a Groq-hosted LLM (e.g., LLaMA 3) with a specific temperature setting.

4. Agent Creation

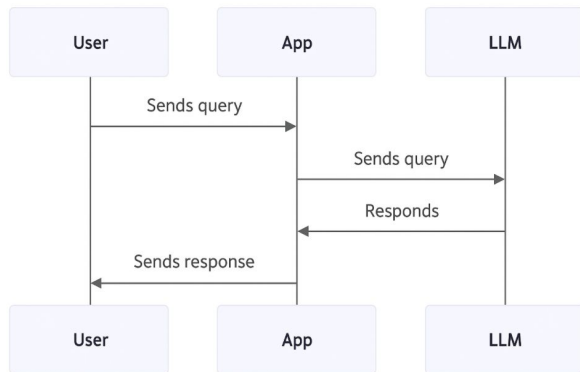
- `create_react_agent()` from `langgraph.prebuilt` sets up an agent capable of reasoning + tool use via ReAct pattern.

5. MCP Client Initialization

- A `ClientSession` is created using `StudioServerParameters()` which connects to a tool server over stdio.

6. Tool Execution & Response Streaming

- The client listens for incoming tool requests, invokes the appropriate tool, and streams back results to the LLM for final output.



Setup Instructions

Create project directory

```
uv init mcp-client
```

```
cd mcp-client
```

Create virtual environment

```
uv venv
```

On Unix or MacOS:

```
source .venv/bin/activate
```

Install required packages

```
uv add mcp anthropic python-dotenv
```

Create our main file

```
touch client.py
```